

Execution Guide

Data-Driven Social Engagement Initiative

Author: Gaurav Ganesh Teegulla | **Course:** Data Science | **Batch:** March 2026

Email: gauravganeshteegulla@gmail.com

SECTION 1 — SYSTEM REQUIREMENTS

Requirement	Specification
Operating System	Windows 10+ / macOS 12+ / Ubuntu 20.04+
Python Version	3.10 or higher (3.11 recommended)
RAM	4 GB minimum — 8 GB recommended
Storage	500 MB free space
Internet Connection	Required for YouTube API calls (optional for dummy mode)
Docker (optional)	Docker Desktop 4.0+ for container deployment

SECTION 2 — PREREQUISITES

macOS Only — Install OpenMP runtime for XGBoost:

XGBoost requires the OpenMP library on Mac. Run this once before setup:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install libomp
```

Windows and Linux users can skip this step. libomp ships with XGBoost on those platforms.

SECTION 3 — SETUP (ONE COMMAND)

Step 1. Extract the source code zip file to your preferred location.

Step 2. Open Terminal (Mac/Linux) or Command Prompt (Windows) and navigate into the project folder:

```
cd "MAJOR project"
```

Step 3. Run the automated setup script:

```
bash setup.sh
```

This script will automatically:

- Create a Python virtual environment in `venv/`
- Install all dependencies listed in `requirements.txt`
- Create required folders: `data/`, `models/`, `logs/`, `reports/`

Step 4. Activate the virtual environment:

```
# macOS / Linux:
source venv/bin/activate

# Windows:
venv\Scripts\activate
```

SECTION 4 — API KEY SETUP (OPTIONAL)

The project runs fully without an API key using built-in realistic dummy data. This step is only needed to connect your own YouTube channel.

Step 1. Copy the environment template:

```
cp .env.example .env
```

Step 2. Open `.env` in any text editor and fill in your credentials:

```
YOUTUBE_API_KEY=your_api_key_here
YOUTUBE_CHANNEL_ID=your_channel_id_here
```

Step 3. To get a free YouTube Data API v3 key, visit console.cloud.google.com → APIs & Services → Enable YouTube Data API v3 → Create Credentials → API Key.

SECTION 5 — RUN THE FULL PIPELINE

Run all 7 stages in sequence with one command:

```
python run_pipeline.py
```

Expected terminal output:

- ▶ Stage 1: Data Extraction
30 videos, 200 comments extracted and saved.
 - ▶ Stage 2: Viral Score Engine
Viral scores computed for 30 videos.
 - ▶ Stage 3: Sentiment Analysis
Sentiment labels assigned to 200 comments.
 - ▶ Stage 4: A/B Testing
4 A/B tests completed.
 - ▶ Stage 5: Virality ML Model
RMSE=11.19, $R^2=0.837$, $CV-R^2=0.936\pm0.025$
 - ▶ Stage 6: Recommendation Engine
7 recommendations generated.
 - ▶ Stage 7: Trend Forecasting
90-day forecast complete. [prophet]
- Pipeline finished.

To run individual stages separately:

```
python run_pipeline.py --stage extract
python run_pipeline.py --stage score
python run_pipeline.py --stage sentiment
python run_pipeline.py --stage ab_test
python run_pipeline.py --stage virality
python run_pipeline.py --stage recommend
python run_pipeline.py --stage forecast
```

SECTION 6 — LAUNCH THE INTERACTIVE DASHBOARD

```
streamlit run src/dashboard/app.py
```

Open your browser and go to: **http://localhost:8501**

The dashboard contains 9 pages accessible from the left sidebar:

Page	Description
1. Overview	KPI cards, top 10 video table, views vs likes scatter
2. Engagement Trends	Monthly bar and area charts for views/likes/comments
3. Viral Score	Leaderboard, violin distribution, per-video gauge chart
4. Sentiment Analysis	Sentiment pie chart, polarity histogram, per-video heatmap
5. Best Posting Time	Day-of-week bars and day × hour heatmap
6. A/B Testing	Pre-built test results + custom test builder with paste input

7. Virality Predictor	Live ML prediction form — enter video details, get a score
8. Recommendations	7 confidence-ranked strategy cards with evidence
9. Trend Forecast	90-day Prophet chart with confidence intervals + keyword velocity

SECTION 7 — VIEW THE AUTO-GENERATED HTML REPORT

After the pipeline runs, a self-contained HTML report is generated automatically:

```
reports/final_report.html
```

Double-click the file to open it in any browser. No server or internet connection is required. All charts are embedded directly inside the HTML file as base64 images.

SECTION 8 — DOCKER DEPLOYMENT (ALTERNATIVE METHOD)

If Docker Desktop is installed, run the entire project in a container:

```
# Build and launch:
docker compose up --build

# Then open: http://localhost:8501

# Stop when done:
docker compose down
```

Docker automatically installs all dependencies and runs the pipeline inside the container. No Python setup needed on the host machine.

SECTION 9 — TROUBLESHOOTING

Error	Fix
<code>ModuleNotFoundError</code>	Run <code>pip install -r requirements.txt</code> again inside the activated venv
XGBoost library load error (Mac)	Run <code>brew install libomp</code> then retry
<code>Prophet import error</code>	Run <code>pip install prophet</code> separately
Dashboard shows blank/empty charts	Run the pipeline first: <code>python run_pipeline.py</code>
Port 8501 already in use	Run <code>streamlit run src/dashboard/app.py --server.port 8502</code>

SECTION 10 — PROJECT FOLDER STRUCTURE

```

MAJOR project/
├─ run_pipeline.py           ← master pipeline runner (entry point)
├─ requirements.txt          ← all Python dependencies
├─ setup.sh                  ← one-shot automated setup script
├─ Dockerfile                ← Docker container definition
├─ docker-compose.yml        ← multi-service Docker config
├─ README.md                 ← full project documentation
├─ config/
│   └─ settings.py           ← central configuration and constants
├─ src/
│   └─ api/
│       ├── youtube_client.py ← YouTube Data API v3 wrapper
│       └─ dummy_data.py      ← realistic dummy data generator
│   └─ pipeline/
│       ├── extract.py        ← Stage 1: data extraction
│       ├── viral_score.py    ← Stage 2: viral score computation
│       └─ report_generator.py ← auto HTML report generation
│   └─ nlp/
│       └─ sentiment.py       ← TextBlob sentiment classifier
│   └─ ml/
│       ├── ab_testing.py     ← Welch's t-test A/B framework
│       ├── virality_model.py ← XGBoost virality predictor
│       ├── recommender.py    ← TF-IDF strategy recommender
│       └─ forecasting.py     ← Prophet trend forecaster
│   └─ dashboard/
│       └─ app.py             ← 9-page Streamlit application
│   └─ utils/
│       ├── db.py             ← SQLite database helper
│       └─ logger.py          ← Loguru rotating logger
├─ data/
│   ├── raw/                  ← videos.csv, comments.csv
│   └─ processed/             ← scored + sentiment enriched CSVs
├─ models/
│   └─ virality_model.joblib  ← saved trained XGBoost model
└─ reports/
    └─ final_report.html      ← auto-generated analytics report

```